# Day 57: Save Account Summary to File

# Day 57: Save Account Summary to File

## 1. Day Number

**Day 57**

## 2. Topic Name

**File Writing with Account Summary**

Today you will practice saving simple account information into a text file.

---

## 3. Connection

You already learned two useful ideas:

- how to work with a `BankAccount` class
- how basic file writing works in Python

Today you will combine those ideas conceptually by saving simple account information into a file.

The basic flow is:

```
Account Data
    ↓
Create formatted text
    ↓
Open file
    ↓
Write text
    ↓
Close file
```

---

## 4. Important Topics

### File write

Python can create or update files using `open()`.

For example:

```
file = open("note.txt", "w")
```

Here:

- `"note.txt"` is the file name
- `"w"` means **write mode**

---

### String formatting

Before saving information, we usually convert the data into readable text.

For example:

```
name = "Amit"
score = 80

text = f"Name: {name}\nScore: {score}"
```

The result looks like:

```
Name: Amit
Score: 80
```

---

### Account data

For today's exercise, the account has only two pieces of information:

```
Account holder name
Balance
```

Example:

```
Account Holder: Rahul
Balance: 5000
```

---

## 5. Foundational Notes

### What is file writing?

Normally, variables disappear when your Python program finishes.

For example:

```
balance = 5000
```

When the program ends, that variable is gone.

But if you save information into a file:

```
account_summary.txt
```

the information remains on your computer.

---

### Opening a file

A common beginner-friendly pattern is:

```
with open("example.txt", "w") as file:
    # write something here
```

`with` is useful because Python automatically closes the file after the block finishes.

---

### Writing into the file

The `write()` method writes text:

```
file.write("Hello")
```

Important:

```
write()
```

expects a **string**.

Therefore, something like a numeric balance may need formatting or conversion.

---

### New lines

Use:

```
\n
```

when you want the next piece of text on another line.

Example:

```
"Name: Rahul\nBalance: 5000"
```

produces:

```
Name: Rahul
Balance: 5000
```

---

### "w" mode

When you use:

```
open("account.txt", "w")
```

Python opens the file in **write mode**.

Be careful: if the file already contains something, `"w"` normally replaces its existing contents.

Later you can learn about append mode:

```
"a"
```

but today's problem only needs `"w"`.

---

## 6. Easy Example

Here is a smaller example unrelated to the final problem.

Suppose we want to save a student's details:

```
student_name = "Ravi"
marks = 75

summary = f"Student: {student_name}\nMarks: {marks}"

with open("student.txt", "w") as file:
    file.write(summary)
```

The generated file would contain:

```
Student: Ravi
Marks: 75
```

Notice the steps:

```
Create data
   ↓
Format data
   ↓
Open file
   ↓
Write formatted data
```

This same idea can be used for an account summary.

---

## 7. Problem Statement

Create:

- an account holder name
- an account balance

Then save both values into a text file as a simple account summary.

For example, your file might contain information in this style:

```
Account Holder: Rahul
Balance: 5000
```

Keep the program very simple.

You do **not** need:

- multiple accounts
- deposits
- withdrawals
- database storage
- complex validation

The goal is only to practice **writing account data into a file**.

---

## 8. Concepts Used

You will use:

- variables
- strings
- numbers
- f-strings/string formatting
- `open()`
- `"w"` mode
- `with`
- `write()`
- `\n`
- account data

Conceptually:

```
Variable
   ↓
Formatted String
   ↓
File
```

---

## 9. Thought Process

Think about the problem in small steps.

### Step 1: What information do I have?

You need:

```
account holder name
balance
```

For example:

```
name → "Rahul"
balance → 5000
```

### Step 2: What should the file contain?

Create a readable account summary.

Something like:

```
Account Holder: Rahul
Balance: 5000
```

### Step 3: How can I create this text?

Use string formatting, such as an f-string.

Think about:

```
label + variable
```

Example idea:

```
"Account Holder: <name>"
```

### Step 4: How do I save it?

Open a text file in write mode.

```
open file
    ↓
write summary
```

### Step 5: How should the file be closed?

Prefer the `with open(...)` approach so Python handles closing automatically.

---

## 10. Pseudocode

```
START

set account holder name

set account balance

create account summary string
    include account holder name
    include balance
    put them on separate lines

open account summary text file in write mode

write summary into file

close file automatically

END
```

Another way to visualize it:

```
name
   \
    → formatted summary → account_summary.txt
   /
balance
```

---

# 11. Suggested Solving Approach

You can solve this in either of two ways.

## Functional/simple approach

For Day 57, this is probably the easiest approach.

```
variables
   ↓
format summary
   ↓
write file
```

You can simply create:

```
name
balance
```

and save them.

---

## OOP-based approach

You can also use a `BankAccount` object.

Conceptually:

```
BankAccount object
       ↓
name + balance
       ↓
formatted account summary
       ↓
text file
```

For example, an account object could contain:

```
holder
balance
```

and your program could use those attributes when creating the file content.

However, you do **not** need to make the OOP version complicated today.

---

# 12. Easy Edge Cases

## Edge Case 1: Balance is 0

Example:

```
Account Holder: Rahul
Balance: 0
```

A balance of `0` is still valid data.

Do not accidentally treat it as missing information.

---

### Edge Case 2: Empty name

Example:

```
name = ""
```

Your program should still be able to write the file.

It might produce:

```
Account Holder:
Balance: 5000
```

For today's beginner exercise, you don't need complex validation.

Later you could check:

```
if name is empty
    show validation message
```

---

## 13. Expected Input

A simple example could be:

```
Account holder name: Rahul
Balance: 5000
```

The values may come from variables directly.

For this exercise, using `input()` is optional unless you want extra practice.

Conceptually:

```
name = Rahul
balance = 5000
```

---

## 14. Expected Output

Suppose the file is called:

```
account_summary.txt
```

After running the program, the file should contain something similar to:

```
Account Summary
Account Holder: Rahul
Balance: 5000
```

Your exact formatting can be slightly different.

The important part is that the **name and balance are successfully saved to the text file**.

---

## 15. Hint Only

Think in three main parts:

```
1. Create account data

2. Build one formatted string
   using the name and balance

3. Open a file using "w"
   and write that string
```

Useful pieces to remember:

```
f"...{variable}..."
```

```
\n
```

```
with open(..., "w") as file:
```

```
file.write(...)
```

Try combining those pieces yourself without adding deposit, withdrawal, or other BankAccount features yet.